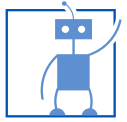


Hinweis zum Urheberrecht

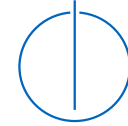
Diese Aufzeichnung und ihre Inhalte sind Eigentum des Lehrstuhls für Robotik, Künstliche Intelligenz und Echtzeitsysteme der Technischen Universität München und werden ausschließlich für angemeldete Studierende des Moduls Kognitive Systeme (IN2222) als Videostream auf der Lernplattform Moodle bereitgestellt. **Download und Weitergabe sind weder ganz noch in Teilen gestattet.**

Copyright Notice

This recording and its contents are property of the Chair of Robotics, Artificial Intelligence and Real-Time Systems of the Technical University of Munich and are provided exclusively for registered students of the module Cognitive Systems (IN2222) as a video stream on the learning platform Moodle. **Download and distribution are prohibited in whole or in part.**



Robotics, Artificial Intelligence and Embedded Systems



Cognitive Systems: Neural Networks and Applied AI

Exercise 8

Cognitive Architectures and ACT-R

Prof. Dr.-Ing. habil. Alois Knoll

Florian Walter, M.Sc.

Erdi Sayar, M.Sc.

Summer Semester

Topics in this Exercise

How to implement a cognitive system?

- Cognitive Architecture – A Definition
- Classification of Cognitive Architectures
- Requirements Analysis
- ACT-R – A Cognitive Architecture

Cognitive Architecture – A Definition

Architecture

Architecture, lat. *architectura*

The art of designing and building of structures and tasteful application of scientific and traditional **rules of good construction** to the materials at hand.

- A unifying or **coherent form or structure**
- A **method** or a **style** of building
- Architectural product or work
- The manner in which the **components** of a computer or computer system are **organized** and **integrated**

From Online Etymology Dictionary and Merriam-Webster

Cognitive Architecture – Definition

Definition 3.1: Cognitive Architecture

A cognitive architecture is the overall, essential structure and process of a broadly-scoped domain-generic computational cognitive model, used for a broad, multiple-level, multi-domain analysis of cognition and behavior. In particular, it deals with componential processes of cognition in a structurally and mechanistically well defined way. Its function is to provide an essential framework to facilitate more detailed modeling and understanding of various components and processes of the mind.

Adapted from Sun (2004)

Unified Theory of Cognition

A cognitive architecture is the **specification** of a cognitive system that defines all of its **components** and the **relations** between them. The design of cognitive architectures was originally motivated in **cognitivist** cognitive science by Newell's concept of a unified theory of cognition:

- A unified theory of cognition addresses a “broad range of cognitive issues, such as attention, memory, problem solving, decision making, and learning” (Vernon, 2014)
- The different issues are covered “from several aspects including psychology, neuroscience, and computer science” (Vernon, 2014)
- All cognitivist cognitive architectures are unified theories of cognition

The instantiation of a cognitive architecture with a specific body of knowledge is called a **cognitive model**

Goals of Cognitive Architectures

Modelling of the Human Mind

- Advance the **understanding** of cognition through concise and **computer-executable** description of **cognitive processes**
- Capturing and reproduction of **experimental data** in models that are not optimal but **realistic**
- Focus on **generality** and **robustness**

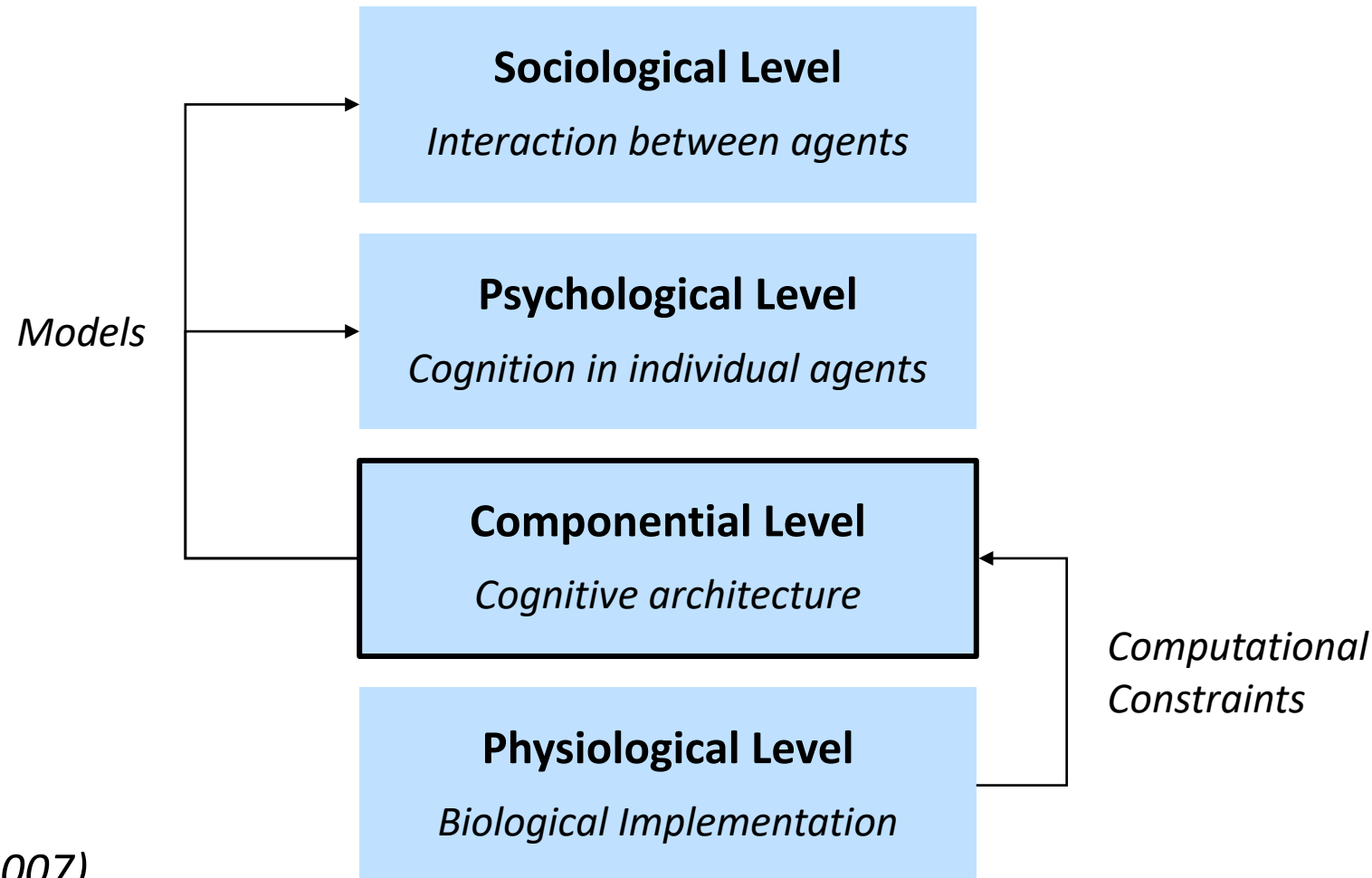
Construction Artificial Cognitive Systems

- Design of artificial systems that are as **capable** as living creatures
- “Brute-force” solutions (search, optimization, big data)
- Focus on **domain-specific** and optimal **methods**

Advantages of Cognitive Architectures

- “Cognitive architectures shed new light on cognition and therefore they are useful tools for advancing the **understanding of human cognition**” (Sun, 2007)
- Cognitive architectures are **generic** and therefore can be applied in different domains (frameworks for cognitive systems, as opposed to **expert systems**)
- Cognitive architectures are **unified models** that **coherently** integrate many different aspects of cognition
- Cognitive architectures are **executable implementations** of theories about cognition that explicitly state all underlying assumptions and hypotheses
- Cognitive architectures are “**deep explanations**” (Sun, 2007) of cognitive processing that go beyond the task-level

Levels of Modeling



Adapted from Sun (2007)

Classification of Cognitive Architectures

Cognitive Architectures History

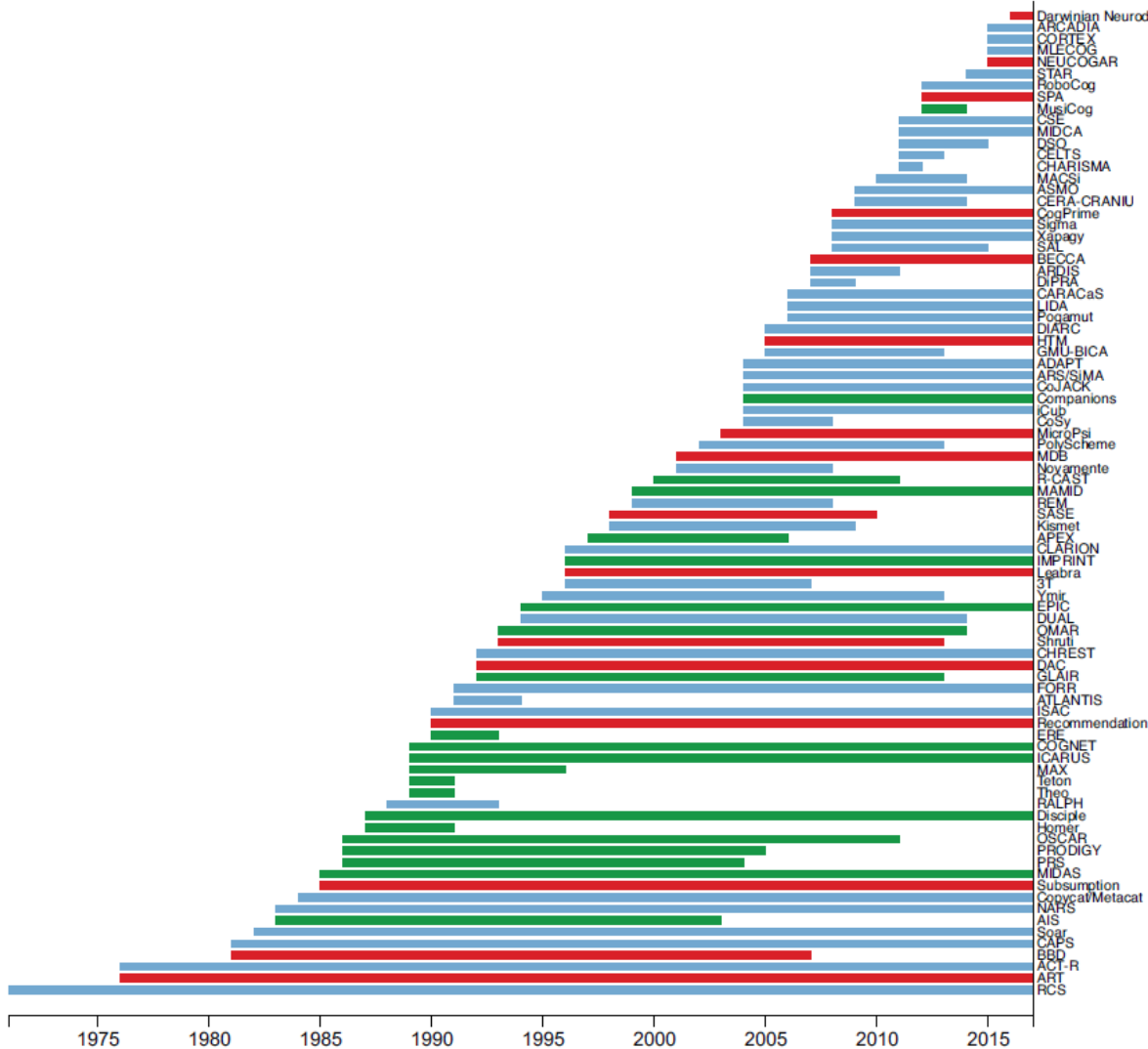


Fig. 2 A timeline of 84 cognitive architectures featured in this survey. Each line corresponds to a single architecture. The architectures are sorted by the starting date, so that the earliest architectures are plotted at the bottom of the figure. Since the explicit beginning and ending dates are known only for a few projects, we recovered the timeline based on the dates of the publications and activity on the project web page or on-line repository. Colors correspond to different types of architectures: symbolic (green), emergent (red) and hybrid (blue). According to this data there was a particular interest in symbolic architectures since mid-1980s until early 1990s, however after 2000s most of the newly developed architectures are hybrid. Emergent architectures, many of which are biologically-inspired, are distributed fairly evenly in the timeline, but remain a relatively small group.

- Symbolic (green)
- Emergent (red)
- Hybrid (blue)

Kotseruba, I & Tsotsos, J.K *Artif Intell Rev* (2018).
<https://doi.org/10.1007/s10642-018-9646-y>

Cognitivist Cognitive Architectures

A cognitivist cognitive architecture models those aspects of cognition that are **independent from the task and **constant** over time**

- It only specifies **basic components** and **processes** such as memory, knowledge representations and learning
- It is a **generic model** that reflects a specific set of **assumptions** about cognitive processing (experimental data, philosophical arguments, hypotheses)
- The architecture is a **framework** that does not include any domain knowledge required for cognitive processing

Production Systems

Reasoning and problem solving in many cognitivist cognitive architectures is implemented by production systems. A production system contains a set of rules of the following form:

if <conditions> **then** <actions>

- The conditions are **propositions** that state properties of and relations among the components of the system; they are usually stored in a **working memory**
- A production is **activated**, i.e. its actions are executed, when all of its conditions are satisfied
- When the conditions of more than one production are satisfied, **precedence rules** determine which of them will be executed
- Production systems are **Turing complete** and can **adapt**

Adapted from the MIT Encyclopedia of the Cognitive Sciences

Emergent Cognitive Architectures

In emergent systems, a cognitive architecture “is the **framework** that facilitates **development**” (Vernon, 2014)

- It models the **phylogenetic configuration** of a newborn agent (i.e. its innate skills) without any requirement for functional modularity
- The **body** is an active part of the architecture
- During ontogeny, the architecture is instantiated through the acquisition of **experience** (as opposed to manually designed knowledge bases in cognitivist cognitive architectures)
- The architecture needs to provide a **structure** that accommodates different cognitive functions (perception, action, learning, etc.)
- As a result of development, the architecture can **change over time**

Requirements Analysis

Desirable Characteristics for Cognitive Architectures

Cognitive Functions

A cognitive architecture is a complex coherent model that needs to support a broad spectrum of different cognitive functions such as:

Perception, categorization, representation, different types of memory, decision-making, reasoning, planning, problem-solving, meta-cognition, communication, action control and execution, different types of learning, motivational processes, goal representation, etc. (Sun, 2007)

- All of these functions are **interdependent**
- The interaction between different functions is **dynamic** and **task-dependent**
- A cognitive architecture must **capture these dynamics**

Design Considerations

Scope (Compactness and Comprehensiveness)

Cognitive architectures are generic and thus also **minimalistic models**. It is therefore not desirable to model all relevant cognitive functions explicitly. The designer of a cognitive architecture must decide which cognitive functions are an **integral** part of the architecture and which other functions can be **implemented later** within the framework of the architecture (**derived capabilities**) (Sun, 2007). Components of the architecture do not necessarily capture an agent's phylogeny (Sun, 2004).

Generality (Simplicity and Degree of Realism)

The generality of a cognitive architectures is an important **tradeoff**. While the modeling of too many specific details makes the architecture **less general**, a too general formulation will make the architecture **under-constrained** and meaningless (Sun, 2007).

Requirements for Cognitive Architectures

Realism

- **Ecological realism**: capability of an agent to engage in everyday activities of its natural environment
- **Bio-evolutionary realism**: a model of human intelligence should be reducible to a model of animal intelligence
- **Cognitive realism**: a cognitive architecture should capture all essential characteristics of human cognition with respect to psychology, philosophy, and neuroscience
- **Eclecticism**: incorporation of results from prior research

Requirements for Cognitive Architectures

Behavioral Characteristics

- **Reactivity**: quick reactions through immediate and fixed responses that do not require elaborate cognitive processing
- **Sequentiality**: activities are executed sequentially step by step over a certain timespan
- **Routineness**: learning, adaption, and execution of reoccurring action patterns by e.g. trail and error

Requirements for Cognitive Architectures

Cognitive Characteristics

- **Explicit processes:** accessible and precise
- **Implicit processes:** inaccessible and imprecise
- **Synergistic interaction:** implicit and explicit processes interact to complement and supplement each other
- **Learning:** implicit and explicit processes interact during top-down learning (explicit learning first, implicit learning later) and bottom-up learning (implicit learning first, explicit learning later)
- **Modularity:** some cognitive faculties are specialized and separated through functional or physical encapsulation

Requirements for Cognitive Architectures

Development

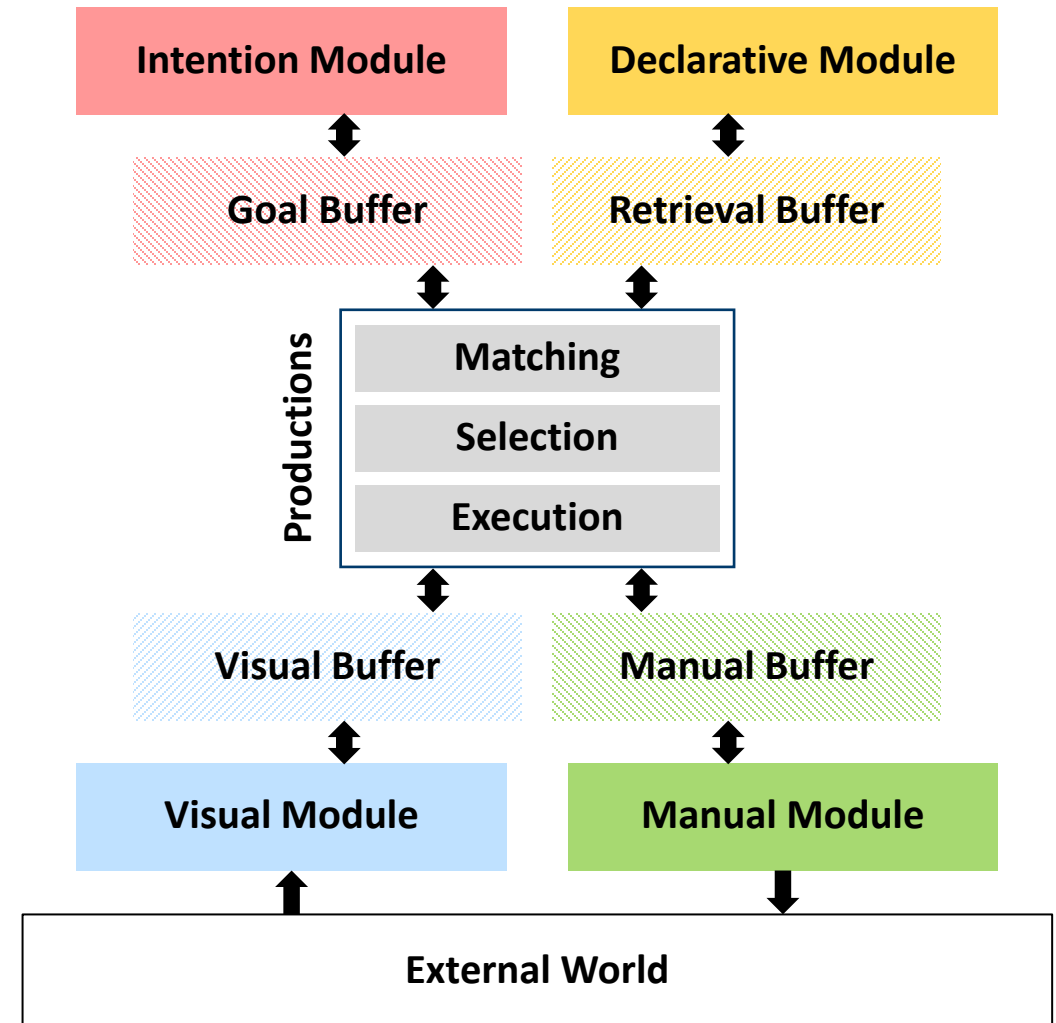
- **Value systems and motives**: guidance during action selection and motivational drive for development (exploration)
- **Physical embodiment**: the body enables to act in the world and to perceive the effects of actions
- **Sensorimotor contingencies**: sensorimotor contingencies store the relations between actions and resulting sensory feedback
- **Perception**: capacity for unsupervised perceptual categorization
- **Constitutive autonomy**: autonomous operation of developmental processes

ACT-R – A Cognitive Architecture

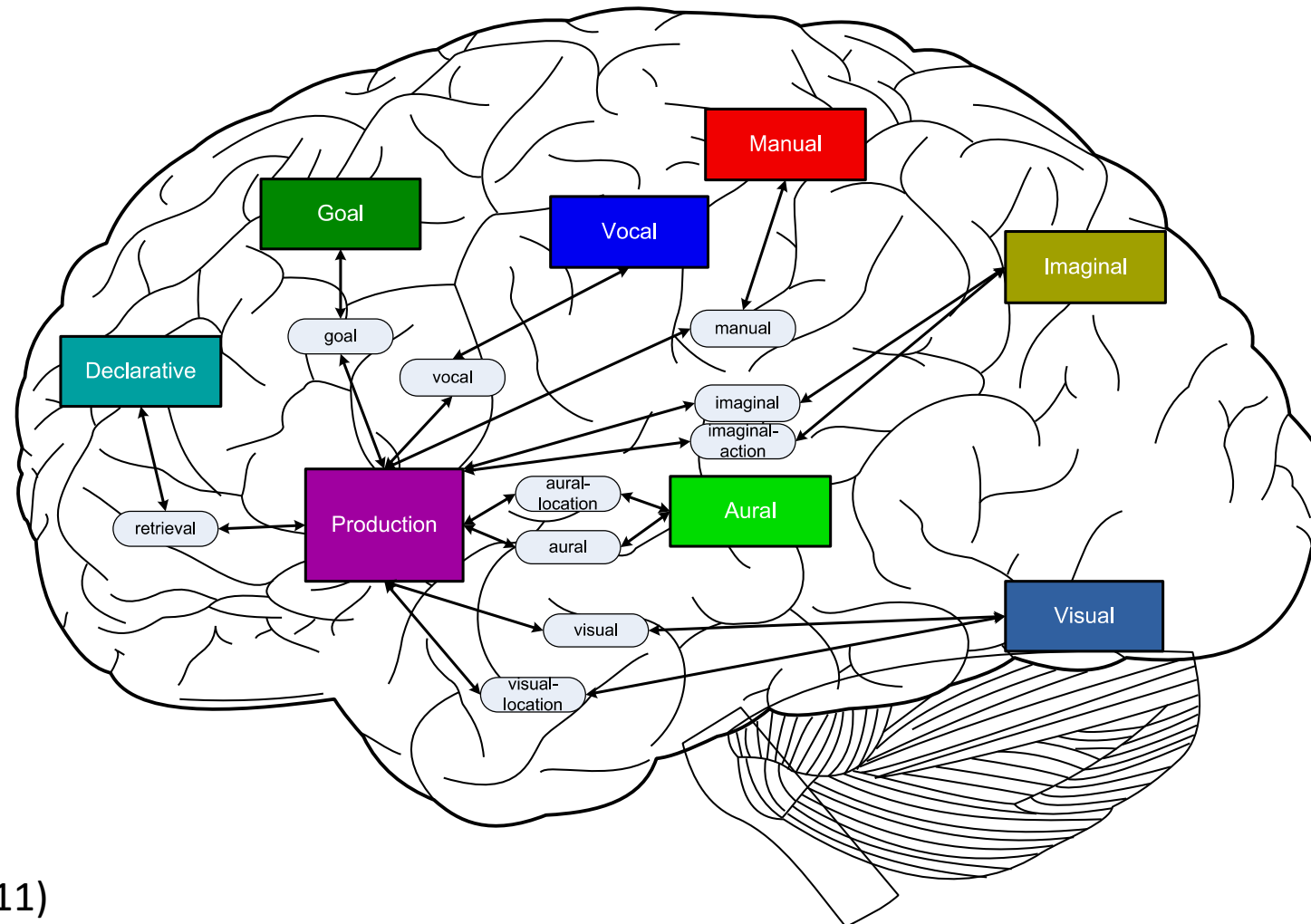
ACT-R

Adaptive control of thought-rational (ACT-R) is a hybrid cognitive architecture and implements the principles of a **unified theory of cognition**. ACT-R is a **software framework** written in Common Lisp.

- ACT-R is based on **modules**, **buffers**, and a **production system**
- While there is no fixed set of modules, the architecture specifies four **main modules** for **intention**, **declarative memory**, **vision**, and **action** and a **procedural module**
- Measurement data from experiments with ACT-R models can be **compared to human data**



Mapping ACT-R Modules to the Brain



From Möbus et al. (2011)

Modules

Modules in ACT-R represent independent **parallel** processing units that **encapsulate specific functions**:

- Every module corresponds to one or more **cognitive functions** of the brain
- Information processing in modules is completely **independent** and separate from other modules
- Every module can encapsulate **arbitrarily** complex sub-functions

Buffers

Information about the current state of the module is exposed through named **buffers**. Every buffer stores one **chunk** and is assigned to exactly **one module**. Buffers serve as interfaces between modules:

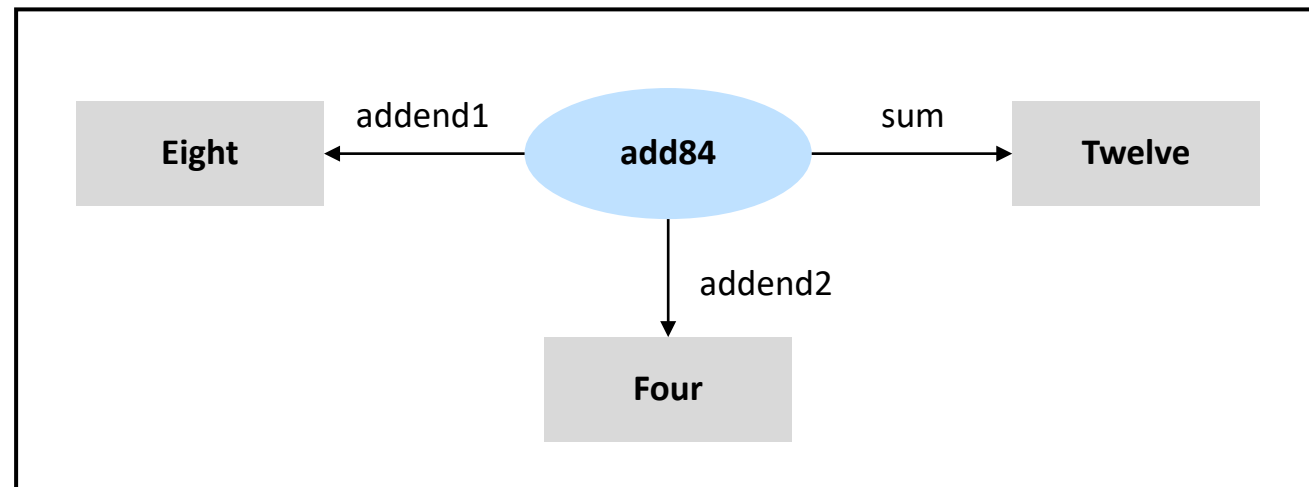
- **Action requests** are submitted through the corresponding module's buffers
- The chunk stored in a buffer is **visible** to all other modules
- Buffers directly process **queries** about their content
 - **Empty**: true if there is no chunk in the buffer
 - **Full**: true if the buffer contains a chunk
 - **Failure**: true if the failure flag is set and the buffer is empty
 - **Requested**: true if the chunk in the buffer is the result of a request
 - **Unrequested**: true if the chunk in the buffer is not the result of a request

Chunks

In ACT-R, declarative knowledge is represented by **chunks** that are comprised of **slots**, each of which stores a single value (which can also be another chunk). Chunks have neither a name nor a type.

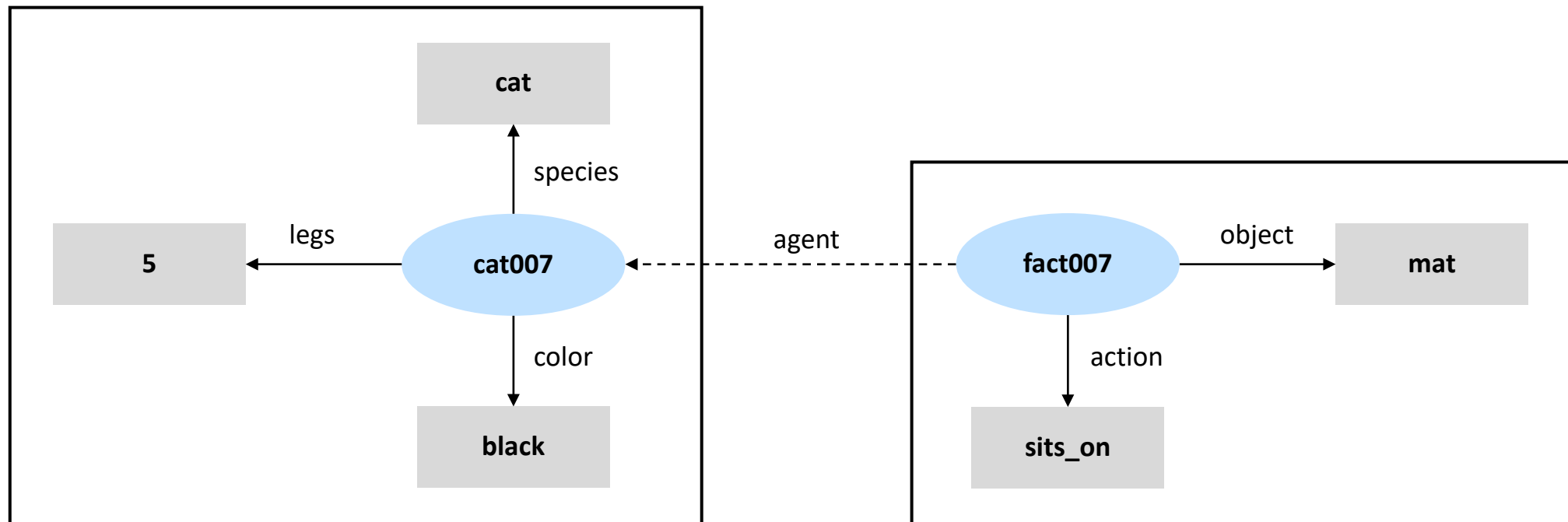
Example: $8 + 4 = 12$

```
add84  
  addend1 Eight  
  addend2 Four  
    sum Twelve
```



Chunks

Example: “The black cat with 5 legs sits on the mat”



Adapted from act-r.psy.cmu.edu/tutorials/ACT-R_intro_tutorial.ppt

Production System

The production system **coordinates** the operation of the individual modules **only** through their buffers:

- Rules in the production system correspond to **procedural knowledge**
- The **conditions** of the rules in the buffer correspond to specific states of the modules
- The production system operates in a **critical cycle** with a length of 50 ms:
 - **Pattern matching**: matching of buffer values with production conditions
 - **Selection** and **execution** of a matching production
 - The **results** of the production are written back to the corresponding module buffers
- Note: the procedural module has no buffer!

Productions

“A production is a **statement** of a particular contingency that **controls behavior**. They can be represented as **if-then rules**” (ACT-R 7 Tutorial). Productions are stored in and managed by the **procedural module** which monitors the states of all module buffers and automatically selects and executes a **matching production**.

- The **condition** of a production is comprised of a set of **buffer tests** that are
 - **patterns** which are matched against the current contents of the buffers
 - **queries** for buffer and module state information
- The **action** of a production is comprised of a set of
 - **buffer changes**
 - **module requests**

Production Syntax

Every production is identified by a name and is assembled from a list of buffer operations. Buffer tests and actions are separated by “==>”:

```
(p ProductionName  
    buffer tests  
==>  
    buffer changes and requests  
)
```

Every buffer operation is declared by a header `mbuffername>` where `m` is a modifier that determines the operation type.

Production Syntax: Buffer Tests

Pattern Matching

Matches the specified chunk pattern against current buffer contents:

```
=buffername>  
  ISA type  
  slot1 const1  
  slot2 =var1
```

Slot values can be either constants (const1) or variables (=var1) that store the value of the matched production. Optionally, a specific slot type can be matched via ISA.

Buffer Condition Queries

Retrieve and check the current buffer and module states:

```
?buffername>  
  buffer <state>  
  
state = full | failure | empty |  
        requested | unrequested
```

```
?buffername>  
  module <state>  
  
state = free | busy | error |  
        etc.
```

Production Syntax: Buffer Changes and Requests

Buffer Modification

Buffer changes modify the content of a buffer's slots:

```
=buffername>  
  ISA type  
  slot1 const1  
  slot2 =var1
```

Assigned values can be either constants (const1) or stored in variables (=var1). Variable names are only defined in the corresponding production. Optionally, a specific slot type can be matched via ISA.

Buffer Clearing

Delete the buffer content:

```
-buffername>
```

Production Syntax: Buffer Changes and Requests

Buffer Requests

Buffer requests trigger module operations. The argument of the request is provided as a chunk. In some situations, requests implicitly trigger a clearing of the buffer.

```
+buffername>  
  ISA type  
  slot1 const1  
  slot2 =var1
```

The ACT-R Main Modules

Visual and Manual Modules

- Visual perception is implemented in a module with **two buffers** that represent the **dorsal** and **ventral** streams of visual information processing
- The manual module implements a simple **hand model** that can operate a virtual **keyboard** and a virtual **mouse**

Intentional Module

- Maintains an internal **context** of intentions for coherent **goal-directed behavior**
- The current goal is exposed via the module buffer
- The only operation supported by the module is the **creation of a new goal chunk**

Declarative Module

The declarative module of the ACT-R architecture implements a storage of chunks that comprises the **declarative memory** of the system. The declarative memory ensures long-term personal and cultural **coherence**.

- Once a **request** is written to the buffer, the module searches the internal memory and returns the chunk that **matches best** in its **retrieval buffer**
- New memories are **collected automatically** from the buffer when a module is cleared; equivalent chunks are **merged**
- The module supports two modes of memory retrieval
 - Purely **symbolic matching** with a random or deterministic selection process for matching chunks
 - **Time-aware** matching and selection based on **sub-symbolic activation dynamics**

Declarative Module: Chunk Activation

Every chunk i in the declarative memory is assigned an **activation** A_i :

$$A_i = B_i + S_i + P_i + \varepsilon_i$$

- **Base-level activation** B_i : recency and frequency of use of the chunk
- **Spreading activation** S_i : effect of the contents of the module buffers
- **Partial matching value** P_i : matching-degree of the chunk for the request
- **Noise** ε_i : noise value with a transient and permanent component

Adapted from the ACT-R 7 Reference Manual

Computing the Activation Value

Base-Level Activation

$$B_i = \ln\left(\sum_{j=1}^n t_j^{-d}\right) + \beta_i$$

- n : Number of presentations (entry into memory or merge with another chunk) of chunk i
- t_j : Time since the j -th presentation
- d : Decay parameter
- β_i : Constant offset

Adapted from the ACT-R 7 Reference Manual

Computing the Activation Value

Spreading Activation

The spreading activation is determined by all buffers k that currently contain a chunk which has a slot storing another chunk and a positive activation value:

$$S_i = \sum_k \sum_j W_{kj} S_{ji}$$

- W_{kj} : Amount of activation from source j in buffer k
- S_{ji} : Association strength from source j to chunk i

Adapted from the ACT-R 7 Reference Manual

Computing the Activation Value

Partial Matching

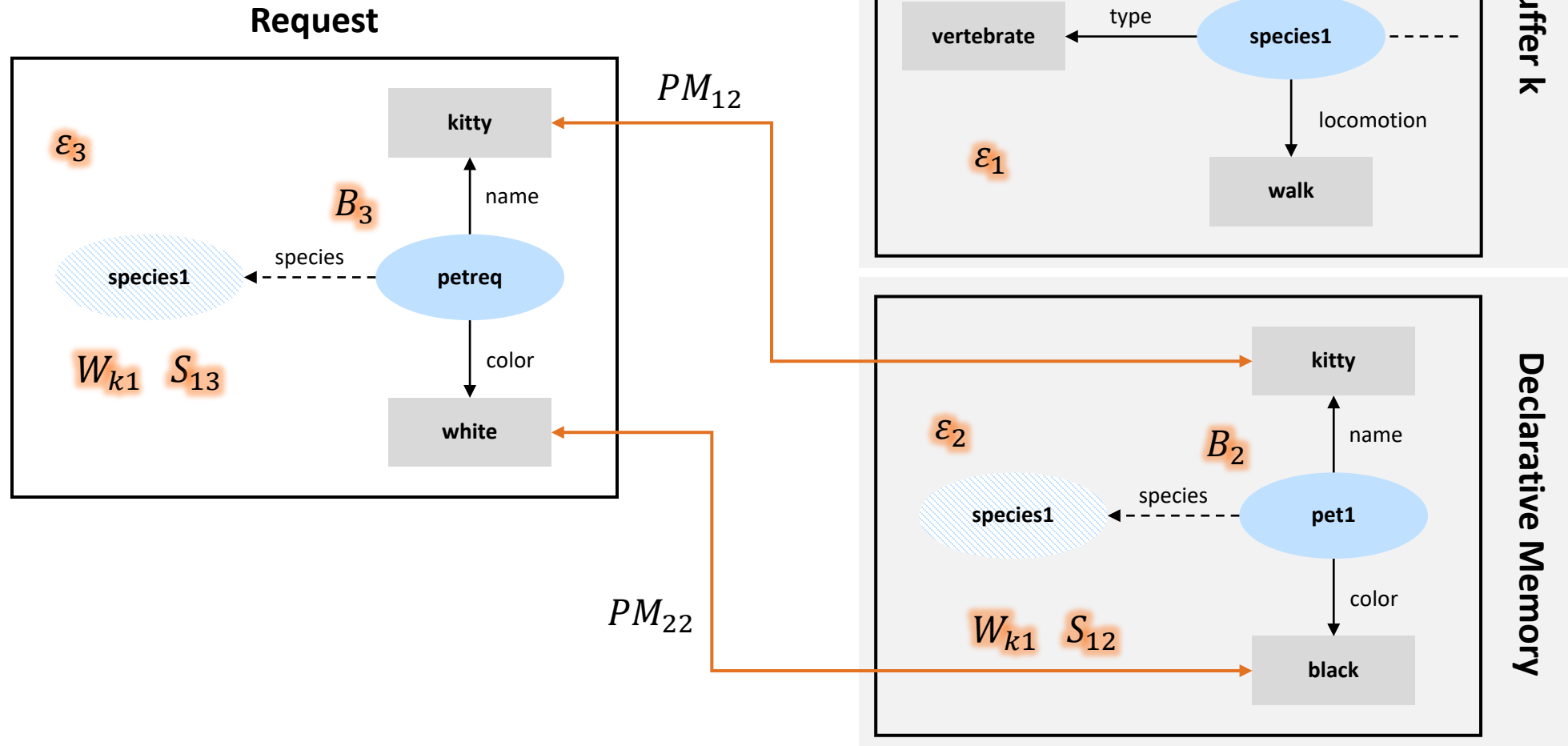
The partial matching is computed by iterating over all slot values k of the retrieval specification:

$$P_i = \sum_k PM_{ki}$$

- P_i : match scale parameter that determines the importance of similarity
- M_{ki} : similarity value between the desired and the actual slot value (usually a negative penalty)

Adapted from the ACT-R 7 Reference Manual

Example



The Procedural Module

The procedural module implements the **procedural memory system** that manages, matches and executes productions. It **coordinates** the interactions between all other modules.

- At every time step, the system computes the **set of viable production rules** by matching the buffer tests against the current buffer contents
- Only **one production** rule can fire in every time step
- If the conditions of more than one production rule are satisfied, a **conflict resolution** mechanism selects the production with the highest **utility value** (or makes a random decision if there are two or more production rules with identical maximum utility value)

Adapted from the ACT-R 7 Reference Manual

Production Utilities

Every production i in ACT-R has a **sub-symbolic utility value** U_i that can be both set manually or learned online during operation:

$$U_i = U_i(n - 1) + \alpha[R_i(n) - U_i(n - 1)]$$

- α : learning rate
- $R_i(n)$: reward for production i for its n -th usage

Adapted from the ACT-R 7 Reference Manual

The Production Compilation Module

ACT-R supports the **learning of new productions** via production compilation.

- The production compilation mechanism tries to combine productions that fire **in sequence**
- Only productions with **compatible buffer usage** can be combined and production compilation is performed on a buffer-by-buffer basis
- Every buffer has a **compilation type** that specifies when a compilation is valid and how the compilation has to be performed
- New productions that are **not semantically equivalent** to existing ones are added to the procedural memory

Adapted from the ACT-R 7 Reference Manual

Example

Adapted from the ACT-R 7 Tutorial

```
(p read-probe
  =goal>
    isa goal
    state attending-probe
  =visual>
    isa visual-object
    value =val
    ?imaginal>
    state free
==>
  +imaginal>
    isa pair
    probe =val
  +retrieval>
    isa pair
    probe =val
  =goal>
    state testing
)
```

```
(p recall
  =goal>
    isa goal
    state testing
  =retrieval>
    isa pair
    answer =ans
    ?manual>
    state free
    ?visual>
    state free
==>
  +manual>
    cmd press-key
    key =ans
  =goal>
    state read-study-item
  +visual>
    cmd clear
)
```



```
(P PRODUCTION0
  =GOAL>
    STATE ATTENDING-PROBE
  =VISUAL>
    VALUE "zinc"
  ?IMAGINAL>
    STATE FREE
  ?MANUAL>
    STATE FREE
  ?VISUAL>
    STATE FREE
==>
  =GOAL>
    STATE READ-STUDY-ITEM
  +VISUAL>
    CMD CLEAR
  +MANUAL>
    CMD PRESS-KEY
    KEY "9"
  +IMAGINAL>
    PROBE "zinc"
)
```

Parallel and Serial Information Processing

Parallel Information Processing

- Information processing **within modules** can be highly parallel to support high-dimensional data streams (perception of the visual field, parallel search in declarative memory)
- Different modules can process data in parallel **independently** from each other

Serial Information Processing

- Every module buffer can store only a **single chunk**
- In each cycle of the production system **exactly one rule** fires

The limitation to a single chunk in every buffer serves as an **attention** mechanism

Example: An ACT-R Model for Counting

Task: Count from num1 to num2

Chunk Types

```
(chunk-type count-order first second)
(chunk-type count-from start end count)
```

Initialize Declarative Memory

```
(add-dm
  (b ISA count-order first 1 second 2)
  (c ISA count-order first 2 second 3)
  (d ISA count-order first 3 second 4)
  (e ISA count-order first 4 second 5)
  (f ISA count-order first 5 second 6)
  (first-goal ISA count-from start 2 end 4))
```

Adapted from the ACT-R 7 Tutorial

Example: An ACT-R Model for Counting

Set the Initial Goal

```
(goal-focus first-goal)
```

Define the Start Production

```
(p start  
  =goal>  
    ISA count-from  
    start =num1  
    count nil  
==>  
  =goal>  
    ISA count-from  
    count =num1  
  +retrieval>  
    ISA count-order  
    first =num1  
)
```

Adapted from the ACT-R 7 Tutorial

Example: An ACT-R Model for Counting

Define the Increment Production

```
(P increment
  =goal>
    ISA count-from
    count =num1
    - end =num1
  =retrieval>
    ISA count-order
    first =num1
    second =num2
==>
  =goal>
    ISA count-from
    count =num2
  +retrieval>
    ISA count-order
    first =num2
)
```

The Stop Production

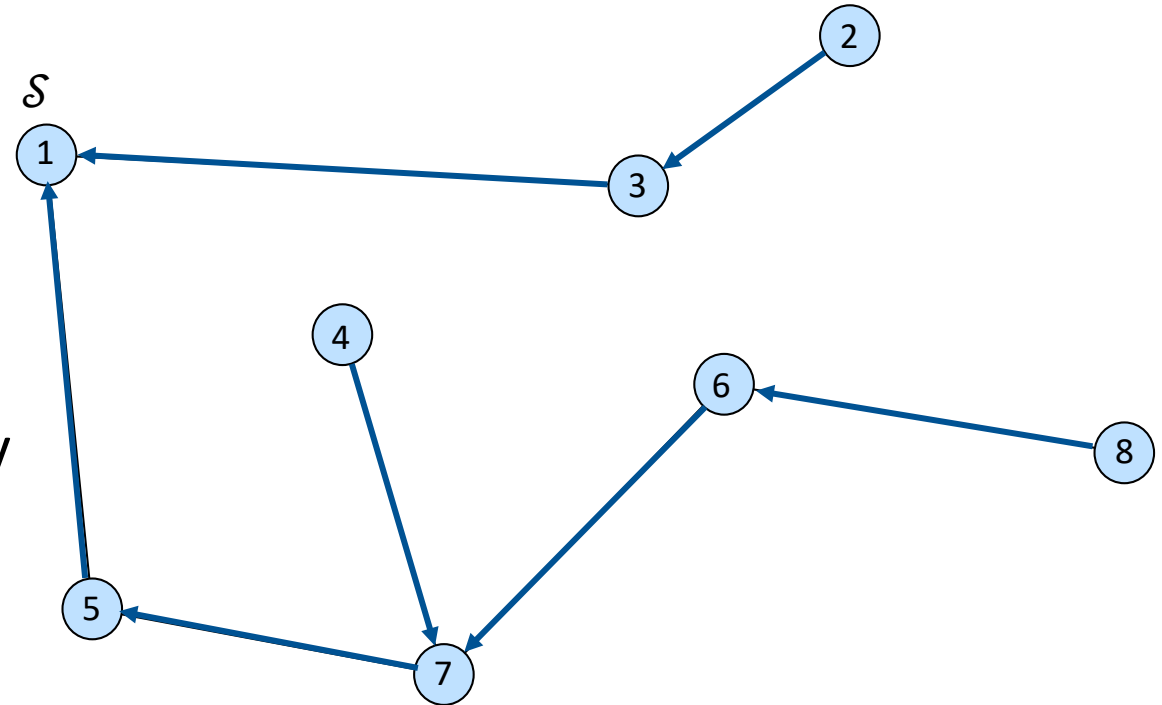
```
(P stop
  =goal>
    ISA count-from
    count =num
    end =num
==>
  -goal>
)
```

Adapted from the ACT-R 7 Tutorial

Modeling with ACT-R

The spanning tree algorithm allows for an easy and efficient computation of shortest paths between the root node $\mathcal{S}$ and any other node in a graph. In this exercise, we will implement a corresponding planner in ACT-R/pyactr (# pip install pyactr)

1. First recall the different types of memory supported by ACT-R. How is memory addressed?
2. Design and implement an ACT-R representation of the graph on the right.
3. Define the productions required to plan from any node of the graph to $\mathcal{S}$.



<http://act-r.psy.cmu.edu/software/>

Literature

[D. Vernon](#), *Artificial cognitive systems: A Primer*. Cambridge, Massachusetts: The MIT Press, 2014.

[R. Sun](#), “Desiderata for cognitive architectures,” *Philosophical Psychology*, vol. 17, no. 3, pp. 341–373, 2004.

[J. R. Anderson](#) *et al.*, “An integrated theory of the mind,” (eng), *Psychological review*, vol. 111, no. 4, pp. 1036–1060, 2004.

[M. Shanahan](#), “A cognitive architecture that combines internal simulation with a global workspace,” (eng), *Consciousness and cognition*, vol. 15, no. 2, pp. 433–449, 2006.

[D. Vernon](#), [C. von Hofsten](#), and [L. Fadiga](#), “The iCub Cognitive Architecture,” in *A Roadmap for Cognitive Development in Humanoid Robots*, Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 121–153.

[R. Sun](#), “The CLARION cognitive architecture: Extending cognitive modeling to social simulation”, in *Cognition and Multi-Agent Interaction*, Cambridge University Press, New York. 2006.